TransRoute: A Novel Hierarchical Transistor-Level Routing Framework Beyond Standard-Cell Methodology

Chen-Hao Hsu[1], David Z. Pan[1], Laurent Perron[2], Frédéric Didier[2], Xiaoqing Xu[3], and Hao Chen[3]
[1]*ECE Department, The University of Texas at Austin, Austin, TX, USA*
[2]*Google Research, Paris, France*
[3]*Google DeepMind, Mountain View, CA, USA*
chenhaoh@utexas.edu, dpan@ece.utexas.edu, {lperron, fdid, xiaoqingxu, haojc}@google.com

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132558

***Abstract*—In advanced technology nodes, benefits from scaling have become limited in terms of power, performance, and area (PPA), necessitating improvements through design-technology co-optimization (DTCO). While the standard-cell methodology is widely used in modern VLSI design, it inherently constrains the potential of DTCO due to its abstraction at the logic level, limiting optimization at the physical level. To bridge this gap, transistor-level design methodology is desired to break the abstraction of standard cells. Existing research has explored large-scale transistor placement, but a gap remains in developing a routing framework that efficiently addresses transistor-level routing challenges. This paper proposes a pioneering transistor-level routing framework for large-scale transistor placements, along with an efficient CP-SAT (Constraint Programming-SATisfiability) formulation for routing in lower layers. Experimental results demonstrate that the proposed routing framework enables transistor-level designs to achieve significantly reduced total wirelength and high utilization rates for designs with hundreds to thousands of transistors, outperforming traditional standard-cell-based approaches in advanced technology nodes.**

I. Introduction

In advanced technology nodes, the PPA benefits from scaling have become limited, highlighting the growing importance of advancements through DTCO. DTCO involves three critical stages: technology, design enablement, and design [1]. Among these, the design enablement stage is critical for bridging technology and design by providing essential inputs to the design phase, including standard cell libraries, intellectual properties (IPs), and signoff environments. Standard cell libraries are fundamental to the VLSI design process due to their scalability, reusability, and seamless integration with electronic design automation (EDA) tools. Each library comprises pre-defined logic functions with fixed physical implementations that standardize the design framework. However, while the standard-cell methodology simplifies design and supports PPA improvement, the fixed physical layouts of standard cells impose constraints on further optimizing the physical design and achieving maximum PPA potential.

To fully unleash the potential of DTCO, it is crucial to move beyond the abstraction of standard cells by adopting a transistor-level physical design approach [2], rather than relying on pre-defined physical implementations in standard cells. Eliminating the abstraction provides greater optimization flexibility, leading to a smaller design area and reduced total wirelength compared to traditional standard-cell-based design. Furthermore, achieving a high utilization rate is challenging with the standard-cell-based approach, as it often struggles to produce LVS/DRC-clean results within a compact design area. A key limitation arises from the fixed physical implementations of standard cells, which hinder the full utilization of lower-layer routing resources (e.g., Metal0) that follow the "use it or waste it" principle. Breaking the standard-cell abstraction allows these lower-layer routing resources to be fully explored and utilized, further enhancing design optimization.

Existing work on transistor-level placement and routing primarily focuses on automating standard cell design. As design rule complexity continues to grow in advanced nodes, manual standard cell design has become increasingly challenging, necessitating advanced automation to address these complexities. Standard cell design automation is categorized into two major approaches: concurrent and sequential. Concurrent approaches [3]–[5] address transistor placement and routing simultaneously for a cell. In contrast, sequential approaches [6]–[10] divide the process into two stages: transistor placement and in-cell routing. Transistor placement [11]–[14] minimizes cell area and enhances routability by leveraging diffusion sharing, while in-cell routing [15]–[17] ensures net connectivity without design rule violations and guarantees pin accessibility. However, existing in-cell routing research primarily focuses on standard-cell-scale transistor-level routing and faces scalability challenges when applied to large-scale placements with hundreds or thousands of transistors.

Recently, TransPlace [18] proposed a transistor-level placement framework that directly places all transistors of a design without standard-cell abstraction, achieving a compact design area and reducing total half-perimeter wirelength. However, existing transistor-level routing approaches are limited to standard-cell scales, typically fewer than 50 transistors, due to scalability issues. To the best of our knowledge, no prior work has addressed the challenges of transistor-level routing for transistor placements with hundreds or even thousands of transistors. Therefore, a new transistor-level routing framework is needed to efficiently route large-scale placements.

In this paper, we present a novel hierarchical transistor-level routing framework capable of routing large-scale transistor placements with high utilization, while optimizing the total wirelength and via count, and handling various complex design rules. The main contributions of this paper are summarized as follows:

- We present the first transistor-level routing framework for a large-scale transistor placement, consisting of three main steps: partitioning, lower-layer routing, and upper-layer routing.
- We propose a transistor partitioning technique that leverages transistor placement results. This approach prioritizes the physical locations of transistors over their logical relationships (those within a standard cell), potentially yielding greater enhancements in PPA.
- We propose an efficient CP-SAT connectivity formulation for transistor-level routing, which controls the flow of each net on a grid-based routing graph while enforcing node exclusiveness through integer encoding at each vertex.
- Experimental results show that the proposed routing framework achieves LVS/DRC-clean transistor-level designs, with an average 22% reduction in wirelength and 14% reduction in via count compared to standard-cell-based designs, while maintaining a high utilization rate of around 90%.

The remainder of this paper is organized as follows. Section II gives the preliminaries and the problem formulation. Section III illustrates our proposed transistor-level routing framework, and Section IV details the proposed CP-SAT formulation for lower-layer routing within a partition. Section V shows the experimental results, and Section VI concludes the paper.

II. Preliminaries and Problem Formulation

This section presents an overview of transistor placement. Then, we formulate the transistor-level routing problem.

A. Transistor Placement

A legal transistor placement requires all transistors of a given transistor-level netlist are placed within a design canvas without overlap. When two transistors that are placed adjacently in the same row, the nets of their shared diffusion must be the same, known as diffusion sharing. Figure 1 shows a legal transistor placement with four standard-cell rows.

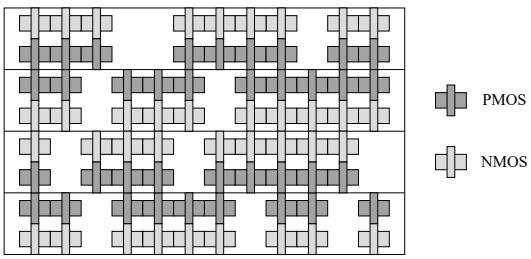

Fig. 1. A transistor placement.

B. Problem Formulation

Problem 1 (Transistor-Level Routing): Given a transistor placement and its corresponding transistor-level netlist, generate a routing solution such that all nets are routed without design rule violations, and the total wirelength and via count are minimized.

III. Transistor-Level Routing Framework

Large-scale transistor-level routing is challenging since we need to consider numerous complex design rules while optimizing total wirelength and via count. The overall flow of our proposed transistor routing consists of three stages as shown in Fig. 2: (1) partitioning, (2) lower-layer routing, and (3) upper-layer routing. First, the layout is partitioned into several partitions based on the transistor placement. From these partitions, we categorize all nets into two types: intra-partition nets and inter-partition nets. An intra-partition net has all its terminals within a single partition, while an inter-partition net has terminals across multiple partitions. Intra-partition nets are routed entirely within their corresponding partitions in the lower routing layers (typically Metal1 and below). For an inter-partition net, we connect its terminals in the lower routing layers and also create an *escape pin* in each of its partitions. These escape pins are then used to connect the inter-partition nets in the upper routing layers (typically Metal1 and above).

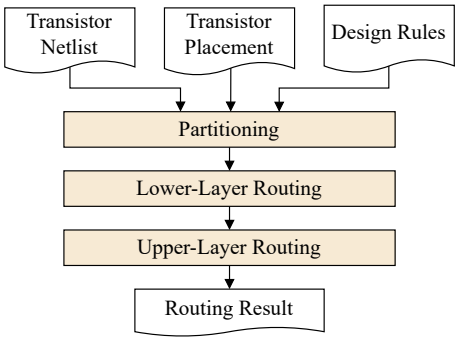

Fig. 2. The overall flow of our proposed transistor-level routing framework.

A. Partitioning

Routing all transistors directly within a macro design can lead to extremely long runtime. To address this challenge, we propose a layout partitioning strategy, where the layout is divided into partitions, each containing a set of transistors. Based on transistor placement, the layout is divided into several partitions according to transistor locations, with each partition being of similar size. Partition size plays a crucial role. If the partitions are too small, they can lead to severe routing congestion in the upper layers. On the other hand, if a partition is too large, connections within the partition may fail. In practice, we use partitions that are 2–4 standard-cell rows by 15–25 contacted poly pitches (CPPs), which are larger than individual standard cells. Furthermore, we use an Euler path (a sequence of diffusion-shared transistors without breaks) as a fundamental component for partitioning, meaning that an Euler path cannot be split across multiple partitions. As a result, partitions are not necessarily rectangular and can take on any rectilinear shape.

Fig. 3 shows the partitioning of the transistor placement in Fig. 1. It is important to emphasize that the partitioning of transistors within a design occurs at the physical level, unlike

the standard-cell methodology, which partitions transistors at the logic level. Such a distinction is crucial as partitioning at the logic level may limit PPA optimization.

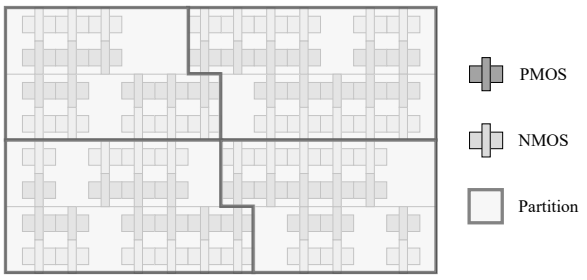

Fig. 3. Four partitions of the transistor placement.

B. Lower-Layer Routing

Lower-layer routing involves net routing for all nets and the creation of escape pins for inter-partition nets within a partition. This stage is particularly challenging due to limited routing resources, the high density of terminals, and complex design rules. Therefore, we formulate lower-layer routing of each partition into a CP-SAT problem and solve the problem by the CP-SAT solver [19], which is detailed in Section IV. Fig. 4 shows the lower-layer routing results of the four partitions in Fig. 3.

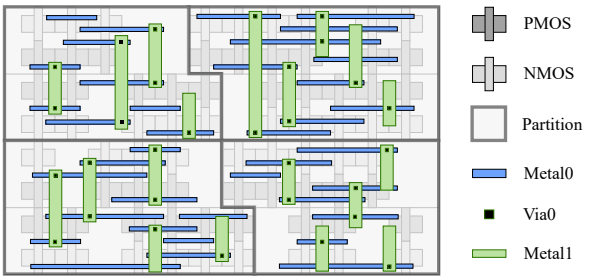

Fig. 4. Lower-layer routing for the four partitions.

C. Upper-Layer Routing

For upper-layer routing, we need to connect all inter-partition nets through their escape pins, with each net guaranteed to have one escape pin in each of its partitions. Since routing resources are more abundant in upper layers, efficiency becomes our primary concern. Connecting all escape pins is similar to the traditional routing problem for standard-cell-based design. Therefore, we could use a conventional router for standard cells to complete the upper-layer routing across all partitions.

IV. CP-SAT Formulation for Lower-Layer Routing

This section details our proposed CP-SAT formulation for lower-layer routing for each partition. We first introduce the grid-based routing graph, followed by the construction of the corresponding flow network. Based on the flow network, we then formulate the CP-SAT constraints and define the objective function.

A. Grid-Based Routing Graph

For lower-layer routing, we use a grid-based routing graph [20]. In advanced technology nodes, routing on each layer is unidirectional with routing directions alternating between layers. Each layer contains multiple tracks, and the grid points on the routing graph represent the intersection points projected from both the layer above and the layer below. Furthermore, due to the non-uniform configuration of tracks, edges in a grid graph can have varying physical lengths on each layer. In this paper, we allow the escape pins created only on the Metal1 layer. Fig. 5 shows a grid-based routing graph from the diffusion/poly layers to the Metal1 layer.

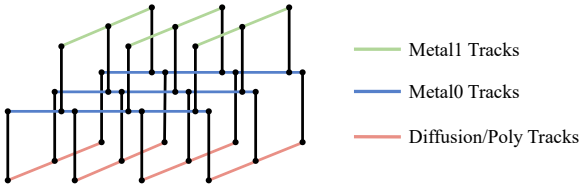

Fig. 5. A grid-based routing graph.

B. Flow Network Construction

To ensure connectivity, we construct a flow network from the grid-based routing graph, with each grid point as a vertex and each edge forming a pair of bidirectional arcs [20]. A terminal pin (diffusion/gate pin) typically has multiple access grid points. A *super-node* is created for each terminal pin. For each net, one of its terminal pins is selected as the source, while the remaining pins are designated as sinks. For the source terminal pin, arcs are constructed from its super-node to each of its access grid points. Conversely, for sink terminal pins, arcs are created from each access grid point to their super-node.

For an inter-partition net, it is necessary to create an escape pin on the escape layer (e.g., Metal1), enabling connections to pins in other partitions. To achieve this, an *escape node* is constructed, and an arc is constructed from each of the grid points on the escape layer to the escape node. Additionally, the escape node is also a sink to each inter-partition net. Fig. 6 shows the flow network based on a routing grid graph.

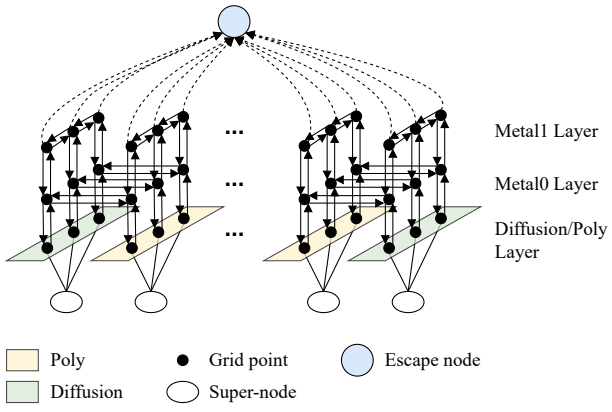

Fig. 6. The flow network based on a grid-based routing graph.

C. Constraints

The CP-SAT constraints for lower-layer routing are divided into two categories: connectivity and design rules. Connectivity constraints ensure the connection of all nets and the generation of an escape pin for each inter-partition net, with these constraints formulated based on a flow network. On the other hand, the design rule constraints are dedicated to preventing any violations of design rules, which are based on an undirected graph derived from the flow network by removing arc directions. Table I summarizes the notations used in this paper.

TABLE I
NOTATIONS USED IN THIS PAPER

Symbol	Description
N	The set of nets in a partition.
N_{inter}	The set of inter-partition nets in a partition.
V/A	The set of vertices/arcs.
V_u^i/V_u^o	The set of vertices where vertex u has an in/out-arc.
s_n	The source node for net n.
T_n	The set of sink nodes for net n.
$t_{n,i}$	The i^{th} sink node for net n.
v_e	The escape node.
$a_{u,v}$	Boolean. Indicate if arc (u,v) is selected.
$f_{u,v}$	Integer. Flow on arc (u,v).
c_u	Integer. The encoding of vertex u.

1) Connectivity: The net routing problem can be formulated as a terminal Steiner forest problem. To construct terminal Steiner forest, we can leverage network flow by controlling the flow between nodes and the flow directions. For a net to be fully connected, there must be a flow from the source node to each of its sink nodes without overlapping with the flows of other nets.

- *Flow Conservation Constraint*: For each vertex in the flow network, the inflow and outflow must be the same for flow conservation:

$$\forall u \in V, \text{inflow}(u) = \text{outflow}(u), \tag{1}$$

where

$$\text{inflow}(u) = \begin{cases} |T_n|, & \text{if } u = s_n, n \in N \\ \sum_{v \in V_u^i} f_{v,u}, & \text{otherwise,} \end{cases}$$
$$\text{outflow}(u) = \begin{cases} 1, & \text{if } u \in T_n \setminus \{v_e\}, n \in N \\ |N_{inter}|, & \text{if } u = v_e \\ \sum_{v \in V_u^o} f_{u,v}, & \text{otherwise.} \end{cases} \tag{2}$$

The inflow of a source node is fixed to the number of sinks, the outflow of each sink node (except the escape node) is fixed to one, and the outflow of the escape node is fixed to the number of inter-partition nets. Note that the domain of each flow variable $f_{u,v}$ is $[0, \max_{n \in N} \deg(n) - 1]$.

- *Selected Arc Constraint*: Arc (u,v) is selected if and only if it has non-zero flow:

$$\forall (u,v) \in A, a_{u,v} \leftrightarrow (f_{u,v} \geq 1). \tag{3}$$

- *In/Out-Arc Count Constraint*: To ensure that each net is connected by a tree, at most one of the in-arcs can be selected for each node, except the escape node:

$$\forall u \in V \setminus \{v_e\}, \sum_{v \in V_u^i} a_{v,u} \leq 1. \tag{4}$$

For each source node, at most one out-arc can be selected to ensure that the source node is a leaf node of a tree:

$$\forall n \in N, \sum_{v \in V_{s_n}^o} a_{s_n,v} \leq 1. \tag{5}$$

- *Vertex and Edge Exclusiveness Constraint*: The flow conservation ensures the connectivity of all nets, but each node or edge can be assigned to at most one net. To enforce vertex exclusiveness, each vertex u has an integer encoding c_u, typically set to the net index (i.e., $1, \ldots, |N|$). The encoding of all the source and sink nodes (except the escape node) is fixed to their net index. Moreover, if an arc (u,v) is selected, then its two vertices must have the same encoding:

$$\forall (u,v) \in A, v \neq v_e, a_{u,v} \rightarrow (c_u = c_v). \tag{6}$$

Note that only node exclusiveness needs to be constrained since edge exclusiveness is implicitly ensured.

Fig. 7 shows an example of terminal Steiner forest construction for two nets, n_1 and n_2, using network flow. Net n_1 includes a source s_1 with three sinks $t_{1,1}$, $t_{1,2}$, and $t_{1,3}$. Similarly, net n_2 has a source s_2 and four sinks $t_{2,1}$, $t_{2,2}$, $t_{2,3}$, and $t_{2,4}$. The inflows of s_1 and s_2 are fixed to 3 and 4 respectively, while the outflow of each sink is fixed to 1.

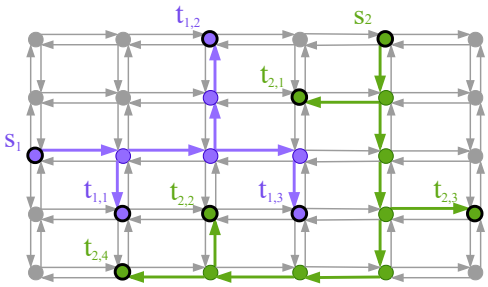

Fig. 7. Terminal Steiner forest construction using network flow.

Our proposed CP-SAT formulation is more efficient than the SAT-friendly integer linear programming (ILP) formulation in [20], which uses only 0-1 integer variables. Unlike [20], which decomposes a k-pin net into $(k-1)$ source-to-sink pairs connected by independent commodity flows, our approach directly generates a Steiner tree by controlling the flow of a net. This allows implicit edge exclusiveness through a single integer variable for node encoding, whereas [20] requires n 0-1 variables per edge, with constraints ensuring at most one is active for edge exclusiveness, where n is the net count.

2) Design Rules: Connectivity constraints ensure all nets are connected with node and edge exclusiveness, but design rules are not addressed. To handle design rules, we add CP-SAT constraints based on the undirected graph derived from a directed graph used for connectivity. Several common design rules are considered, such as minimum area, end-of-line spacing, via spacing, etc. Based on the undirected graph, we can formulate CP-SAT constraints for the design rules, following a similar approach to that in the works [3], [20].

D. Objective

The objective of lower-layer routing is to minimize the total wirelength and via costs. Each arc in the routing graph, excluding those connected to super-nodes or the escape node, represents either a wire or a via. Wire costs are calculated as the product of wirelength and a layer-specific unit cost, while via costs are determined by a user-defined parameter. Thus, the objective in the CP-SAT formulation is $\sum_{a \in A} \text{cost}(a)$.

V. Experimental Results

The proposed partitioning and lower-layer routing methods were implemented in the C++ programming language with the CP-SAT solver [19]. For upper-layer routing, we used the OpenROAD project [21] and the Cadence Innovus tool [22]. This section describes the benchmark generation process and provides a detailed comparison of standard-cell-based designs versus transistor-level designs, along with an evaluation of the efficiency of the proposed CP-SAT connectivity formulation.

A. Benchmarks

Our transistor-level netlists are generated from standard-cell-based designs. First, we synthesize an RTL design using a standard-cell library to obtain a gate-level netlist. The gate-level netlist is then flattened by replacing each standard cell with its corresponding SPICE transistor-level netlist. The resulting transistor-level netlist includes all transistors from the original standard-cell-based design, but with the hierarchy fully eliminated.

Table II gives the benchmark statistics, where "Tech.", "#Trans.", "#Net", "#SDC", and "SDC Area" denote the technology, transistor count, signal net count, standard cell (SDC) count, and total standard cell area (μm^2), respectively. To evaluate the proposed transistor-level routing framework, there are totally eight designs across two advanced industry technology nodes: Tech1 (Design1 to Design4) and Tech2 (Design5 to Design8).

TABLE II
Benchmark statistics

Benchmark	Tech.	#Trans.	#Net	#SDC	SDC Area
Design1	Tech1	72	40	15	0.41
Design2	Tech1	162	90	23	0.87
Design3	Tech1	666	300	76	3.38
Design4	Tech1	684	314	84	3.51
Design5	Tech2	286	129	28	1.14
Design6	Tech2	633	282	55	2.49
Design7	Tech2	656	336	77	2.68
Design8	Tech2	1969	936	180	7.72

B. Overall Quality Comparison

To evaluate transistor-level routing, we compare the post-routing results of transistor-level designs with their corresponding standard-cell-based designs. Two different placement and routing flows are used for the standard-cell-based designs, depending on the technology node. The open-source OpenROAD [21] design flow is used for Tech1 (Design1 to Design4), while the commercial Cadence Innovus flow is employed for Tech2 (Design5 to Design8). Furthermore, transistor-level placements for each design are generated using a placement framework based on TransPlace [18]. All the transistor-level designs are LVS/DRC-clean.

Table III summarizes a comprehensive comparison of the post-routing results, where "Total WL", "Total Via", "Design Area", and "Util." represent the total wirelength (μm), via count, design area (μm^2), and utilization rate, respectively. The design area refers to the bounding box of all standard cells or transistors, and the utilization rate, equivalent to that in standard-cell-based designs, is calculated by dividing the total SDC area by the design area. The transistor-level designs consistently exhibit significantly less total wirelength and via count compared to the standard-cell-based designs, with an average wirelength reduction of 23% for Tech1 and 21% for Tech2, and via count reductions of 25% for Tech1 and 2% for Tech2. These improvements can be attributed to two key factors. First, transistor-level placement, without the constraints of standard-cell abstraction, allows for more diffusion sharing opportunities, enabling designs to fit into a smaller area and reducing the overall HPWL. Second, the absence of standard-cell abstraction enables more nets to be routed in the lower layers, minimizing the need for detours into upper layers. Furthermore, transistor-level designs achieve significantly smaller design areas, with reductions of 29% for Tech1 and 31% for Tech2. As a result, these designs reach a high utilization rate of around 90%, which is challenging to achieve with DRC-clean results in standard-cell-based designs.

C. Wirelength Breakdown

Fig. 8 presents the breakdown of wirelength by layers for both the standard-cell-based design and transistor-level (T-level) design of Design3, Design4, Design7, and Design8. For Design3 and Design4, the transistor-level designs show significantly reduced wirelength in the upper layers (Metal2 and above), while the lower layers have slightly more wirelength compared to the standard-cell-based designs. However, the total wirelength in the transistor-level designs remains lower. It is important to note that lower-layer routing resources follow the "use it or waste it" principle. In standard-cell-based designs, routing on Metal0 is restricted due to fixed routing within standard cells. In contrast, transistor-level designs can fully leverage the lower-layer routing, as they are not constrained by the fixed physical implementations of standard cells. For Design7 and Design8, the transistor-level designs have significantly less wirelength across all layers, except for a slight increase in Metal3. This minor increase helps avoid congestion in lower layers due to the smaller design area compared to standard-cell-based designs.

TABLE III
OVERALL COMPARISON BETWEEN STANDARD-CELL-BASED DESIGNS AND TRANSISTOR-LEVEL DESIGNS

Benchmark	Standard-Cell-Based Design				Transistor-Level Design			
	Total WL	Total Via	Design Area	Util.	Total WL	Total Via	Design Area	Util.
Design1	15.71	67	0.81	0.51	8.79	26	0.41	1.00
Design2	36.37	150	1.38	0.63	26.50	100	0.97	0.89
Design3	167.76	618	4.70	0.72	155.40	609	3.85	0.88
Design4	181.20	646	4.79	0.73	153.80	611	3.76	0.93
Avg. Ratio	1.00	1.00	1.00	-	0.77	0.75	0.71	-
Design5	54.26	229	1.75	0.65	37.70	191	1.18	0.97
Design6	115.11	500	4.09	0.61	86.21	473	2.76	0.90
Design7	160.74	683	4.19	0.64	129.84	652	2.92	0.92
Design8	506.49	1907	11.83	0.65	451.83	2263	8.61	0.90
Avg. Ratio	1.00	1.00	1.00	-	0.79	0.98	0.69	-

TABLE IV
COMPARISON OF CONNECTIVITY FORMULATIONS

Benchmark	Grid Sizes	#Nets	#Nodes	Max Deg.	Time Limit (s)	[20]		Ours	
						Feasible Time (s)	Length	Feasible Time (s)	Length
Graph1	8×8×3	10	27	5	60	4.20	84.30	1.03	81.70
Graph2	10×10×4	10	32	7	120	16.52	140.20	14.67	118.90
Graph3	10×10×3	15	35	3	600	236.61	144.00	30.37	130.60
Graph4	15×15×4	12	37	7	900	260.99	295.50	181.58	196.20
Graph5	12×12×3	20	40	2	1200	1055.76	279.00	8.71	203.20
Avg. Ratio						1.00	1.00	0.39	0.82

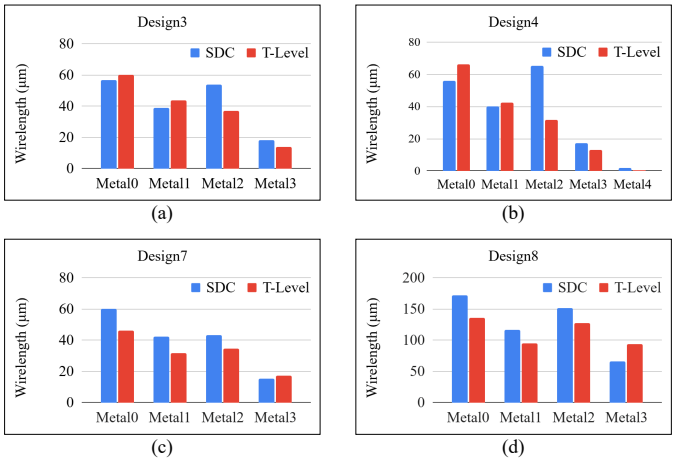

Fig. 8. Wirelength breakdown by layers: (a) Design3, (b) Design4, (c) Design7, and (d) Design8.

D. Connectivity Formulation Comparison

To evaluate the efficiency of our proposed CP-SAT connectivity formulation, we implemented both the SAT-friendly ILP formulation [20] and our CP-SAT formulation using the CP-SAT solver. We generated five synthetic graphs with various grid sizes and net counts for terminal Steiner forest construction, as given in Table IV, where "#Nets", "#Nodes", and "Max Deg." represent the net count, total node count, and maximum net degree, respectively. A time limit was set for each graph based on the problem size. The CP-SAT solver finds an initial feasible solution and continues optimizing until it reaches an optimal solution or the time limit. Due to the solver's non-deterministic nature, each formulation was run 10 times per graph to measure the average time to find the first feasible solution and the average total length.

Table IV compares the performance of the SAT-friendly ILP and CP-SAT formulations. For graph4 and graph5, the solver timed out in 4 and 6 out of 10 runs, respectively, without finding a feasible solution when using the SAT-friendly ILP formulation. On average, the proposed formulation finds the first feasible solution 61% faster and reduces total net length by 18% within the time limit compared to the SAT-friendly ILP formulation. The results demonstrate that the proposed CP-SAT formulation allows the solver to find a feasible solution more efficiently, enabling it to achieve a better-optimized solution within the given time limit.

VI. CONCLUSION

In this paper, we have presented an innovative transistor-level routing framework capable of routing all nets within large-scale transistor placements. The layout partitioning method has been proposed to accurately reflect the proximity of transistors at the physical level, rather than relying on logic-level relationships, facilitating the optimization of the ultimate layout. Furthermore, our proposed CP-SAT-based lower-layer routing algorithm efficiently routes all nets with a novel connectivity formulation, optimizing both wirelength and via costs while handling complex design rules in cutting-edge technology nodes. Experimental results have demonstrated the transistor-level design's significant improvements over standard-cell-based designs in wirelength, via count, and design area. Transistor-level routing is the final step toward achieving complete transistor-level design for macro designs, and this work marks the first attempt to fully realize this goal. As DTCO becomes increasingly important in advanced technology nodes, we believe that the transistor-level design approach, beyond standard-cell methodology, can fully exploit the potential of DTCO.

REFERENCES

[1] S. Choi, J. Jung, A. B. Kahng, M. Kim, C.-H. Park, B. Pramanik, and D. Yoon, "PROBE3.0: A systematic framework for design-technology pathfinding with improved design enablement," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 2023.

[2] R. Hentschke, "Physical design at the transistor level beyond standard-cell methodology," in *Proceedings of ACM International Symposium on Physical Design (ISPD)*, 2022, pp. 141–143.

[3] D. Lee, D. Park, C.-T. Ho, I. Kang, H. Kim, S. Gao, B. Lin, and C.-K. Cheng, "SP&R: SMT-based simultaneous place-and-route for standard cell synthesis of advanced nodes," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 40, no. 10, pp. 2142–2155, 2020.

[4] C.-K. Cheng, C.-T. Ho, D. Lee, and B. Lin, "Multirow complementary-FET (CFET) standard cell synthesis framework using satisfiability modulo theories (SMTs)," *IEEE Journal on Exploratory Solid-State Computational Devices and Circuits*, vol. 7, no. 1, pp. 43–51, 2021.

[5] D. Lee, C.-T. Ho, I. Kang, S. Gao, B. Lin, and C.-K. Cheng, "Many-tier vertical gate-all-around nanowire FET standard cell synthesis for advanced technology nodes," *IEEE Journal on Exploratory Solid-State Computational Devices and Circuits*, vol. 7, no. 1, pp. 52–60, 2021.

[6] P. Van Cleeff, S. Hougardy, J. Silvanus, and T. Werner, "BonnCell: Automatic cell layout in the 7-nm era," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 39, no. 10, pp. 2872–2885, 2019.

[7] Y.-L. Li, S.-T. Lin, S. Nishizawa, H.-Y. Su, M.-J. Fong, O. Chen, and H. Onodera, "NCTUcell: A DDA-aware cell library generator for FinFET structure with implicitly adjustable grid map," in *Proceedings of ACM/IEEE Design Automation Conference (DAC)*, 2019, pp. 1–6.

[8] H. Ren and M. Fojtik, "NVCell: Standard cell layout in advanced technology nodes with reinforcement learning," in *Proceedings of ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 1291–1294.

[9] C.-T. Ho, A. Ho, M. Fojtik, M. Kim, S. Wei, Y. Li, B. Khailany, and H. Ren, "NVCell 2: Routability-driven standard cell layout in advanced nodes with lattice graph routability model," in *Proceedings of ACM International Symposium on Physical Design (ISPD)*, 2023, pp. 44–52.

[10] C.-T. Ho, A. Chandna, D. Guan, A. Ho, M. Kim, Y. Li, and H. Ren, "Novel transformer model based clustering method for standard cell design automation," in *Proceedings of ACM International Symposium on Physical Design (ISPD)*, 2024, pp. 195–203.

[11] A. Sorokin and N. Ryzhenko, "SAT-based placement adjustment of FinFETs inside unroutable standard cells targeting feasible DRC-clean routing," in *Proceedings of ACM Great Lakes Symposium on VLSI (GLSVLSI)*, 2019, pp. 159–164.

[12] M. Cardoso, A. Bubolz, J. Cortadella, L. Rosa, and F. Marques, "Transistor placement for automatic cell synthesis through boolean satisfiability," in *Proceedings of IEEE International Symposium on Circuits and Systems (ISCAS)*, 2020, pp. 1–5.

[13] K. Jo and T. Kim, "Optimal transistor placement combined with global in-cell routing in standard cell layout synthesis," in *Proceedings of IEEE International Conference on Computer Design (ICCD)*, 2021, pp. 517–524.

[14] K. Baek and T. Kim, "CSyn-fp: Standard cell synthesis of advanced nodes with simultaneous transistor folding and placement," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 2023.

[15] N. Ryzhenko and S. Burns, "Standard cell routing via boolean satisfiability," in *Proceedings of ACM/IEEE Design Automation Conference (DAC)*, 2012, pp. 603–612.

[16] J. Cortadella, J. Petit, S. Gomez, and F. Moll, "A boolean rule-based approach for manufacturability-aware cell routing," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 33, no. 3, pp. 409–422, 2014.

[17] Y.-L. Li, S.-T. Lin, S. Nishizawa, and H. Onodera, "Mcell: multi-row cell layout synthesis with resource constrained max-sat based detailed routing," in *Proceedings of IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2020, pp. 1–8.

[18] C.-H. Hsu, X. Xu, H. Chen, D. Ruic, and D. Z. Pan, "TransPlace: A scalable transistor-level placer for VLSI beyond standard-cell-based design," in *Proceedings of IEEE/ACM Asia and South Pacific Design Automation Conference (ASPDAC)*, 2024, pp. 312–318.

[19] L. Perron and F. Didier. CP-SAT. Google. [Online]. Available: https://developers.google.com/optimization/cp/cp_solver/

[20] D. Park, D. Lee, I. Kang, C. Holtz, S. Gao, B. Lin, and C.-K. Cheng, "Grid-based framework for routability analysis and diagnosis with conditional design rules," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 39, no. 12, pp. 5097–5110, 2020.

[21] T. Ajayi and D. Blaauw, "OpenROAD: Toward a self-driving, open-source digital layout implementation tool chain," in *Proceedings of Government Microcircuit Applications and Critical Technology Conference*, 2019, pp. 1–6.

[22] Innovus implementation system. Cadence. [Online]. Available: https://www.cadence.com/en_US/home/tools/digital-design-and-signoff/soc-implementation-and-floorplanning/innovus-implementation-system.html